Milestone 2 Detailed Blueprint

Face Verification System - complete implementation plan, file placement guide, function catalog, and dependency map

Purpose of this document: give you a build order that is concrete enough to implement the milestone without overengineering. It explains which files belong in **src/** versus **scripts/**, what each function should do, why that function is needed, and which other functions or scripts will call it.

1. What this milestone is really asking for

- Start from your reproducible Milestone 1 verifier and turn it into a **repeatable evaluation system**.
- Run the same system multiple times under controlled changes, especially threshold changes and at least one **data-centric improvement**.
- Track each important run so a grader can see what changed, what data version was used, and what result was produced.
- Choose the operating threshold on a **non-test split**, then lock it before reporting final test performance.
- Study failure modes with **error analysis**, not only a single accuracy number.
- Add validation and tests so the pipeline behaves reliably from a clean clone.

2. Recommended repository layout

Use this structure. It cleanly separates reusable logic from command-line entry points.

```
project/
README.md
configs/
  pairs_baseline.yaml
  baseline_val_sweep.yaml
  baseline_val_locked.yaml
  baseline_test.yaml
  pairs_improved.yaml
  improved_val_sweep.yaml
  improved_test.yaml
scripts/
  prepare_pairs.py
  run_eval.py
  run_error_analysis.py
src/
  config.py
  io_utils.py
  validation.py
  metrics.py
  thresholding.py
  evaluation.py
  tracking.py
  plotting.py
  error_analysis.py
tests/
  test_metrics.py
  test_thresholding.py
  test_validation.py
  test_integration_eval.py
data/processed/
```

```
outputs/runs/
reports/
```

Why this split matters

- **src/** contains reusable functions and modules. These should be importable and testable.
- **scripts/** contains the top-level commands you actually run from the terminal.
- **configs/** stores experiment settings so runs are easy to reproduce.
- **outputs/** stores run artifacts and logs.
- **tests/** proves the system behaves correctly on small controlled cases.

Rule of thumb: if something is reusable by multiple scripts, it belongs in **src/**. If something is the actual entry point you run, it belongs in **scripts/**.

3. Step-by-step build order

- **Step A - Freeze baseline artifacts:** keep one deterministic pair version from Milestone 1 or regenerate it deterministically and save it.
- **Step B - Build the reusable evaluation core:** metrics, thresholding, validation, tracking, and evaluation helpers.
- **Step C - Build run_eval.py:** one script that loads config, runs evaluation, saves artifacts, and appends a tracked run.
- **Step D - Execute baseline runs:** validation sweep, locked-threshold evaluation, and final test evaluation.
- **Step E - Make one data-centric improvement:** quality filtering, pair balancing, duplicate removal, identity capping, or another deterministic pair-policy change.
- **Step F - Execute improved runs:** repeat the same evaluation discipline after the data-centric change.
- **Step G - Error analysis:** extract false positives, false negatives, and at least two error slices with examples and hypotheses.
- **Step H - Reliability:** add validation checks and small tests.
- **Step I - Packaging:** write the report, update README, run a clean-clone test, and tag the reproducible commit.

4. Full system flow

```
prepare_pairs.py -> saves deterministic pair artifact(s) -> run_eval.py loads pair artifact ->
validates input -> computes or loads scores -> runs threshold sweep or fixed-threshold evaluation ->
saves metrics / confusion matrix / plot / predictions -> appends a run row to run_summary.csv ->
optional run_error_analysis.py loads saved predictions and builds slices for the report.
```

5. scripts/ - the files you actually run

These files should stay thin. They orchestrate work by calling functions from src/ instead of containing all logic themselves.

5.1 scripts/prepare_pairs.py

This script is responsible for generating a deterministic pair artifact. You do **not** want run_eval.py to regenerate pairs every time, because then each evaluation could accidentally change more than one thing.

Function / Location	What it does	Why it exists	Used by
main() scripts/prepare_pairs.py	Parses config, calls pair-preparation functions, saves pair CSV and metadata.	Provides one reproducible command to create a named pair version.	Called from terminal; uses load_config(), build_split_map(), build_pairs_df(), save_pairs_artifacts().
build_split_map() can live in script or src later	Creates deterministic identity or sample split assignment.	Ensures train / val / test are fixed and reproducible.	Used by build_pairs_df().
build_pairs_df()	Constructs positive and negative pairs and attaches labels and split names.	The whole milestone depends on evaluating saved pairs, not ad hoc pairs.	Used by save_pairs_artifacts(), validation functions, and later by run_eval.py after loading.
save_pairs_artifacts()	Writes pairs_v1.csv or pairs_v2.csv plus lightweight metadata about the policy.	You need a traceable pair version for fair baseline vs improved comparison.	Used by main(); outputs are later consumed by run_eval.py.

Output of prepare_pairs.py

- A saved CSV such as **data/processed/pairs_v1.csv** with columns like pair_id, img1_path, img2_path, label, split.
- Optional metadata JSON explaining the pair policy, filtering rules, seed, and version label.
- A second version such as **pairs_v2.csv** only when you intentionally make your data-centric change.

5.2 scripts/run_eval.py

This is the heart of Milestone 2. One execution of run_eval.py should correspond to one tracked run.

Function / Location	What it does	Why it exists	Used by
main() scripts/run_eval.py	Parses args, loads config, creates run folder, loads pairs, validates data, gets scores, runs evaluation mode, saves outputs, appends summary.	Turns the system into a repeatable experiment runner.	Called from terminal; uses almost every core module in src/.
parse_args()	Reads --config and optional --note.	Lets you control runs without editing code.	Used only by main().
print_run_summary()	Prints the final run id, threshold, metrics, and output path.	Makes the terminal output readable and helps debugging.	Used by main() at the end.

What run_eval.py should never do by default

- It should not randomly regenerate splits.
- It should not regenerate pairs every time unless the config is explicitly for a new pair version you intentionally created.
- It should not silently tune thresholds on the test split.
- It should not mix tracking logic with metric formulas; those belong in src/.

5.3 scripts/run_error_analysis.py

This script is optional but highly recommended because the milestone wants defined error slices with examples and hypotheses.

Function / Location	What it does	Why it exists	Used by
main() scripts/run_error_analysis.py	Loads a saved predictions file from a tracked run and produces slice outputs and example lists.	Keeps analysis separate from evaluation so the evaluation runner stays clean.	Called from terminal; uses load_predictions(), build_error_slices(), save_slice_reports().
load_predictions()	Loads predictions.csv from a run folder.	Error analysis should operate on exactly the run you are reporting.	Used by main().
save_slice_reports()	Writes slice summaries and representative example tables.	You need concrete evidence for the report.	Used by main(); outputs appear in the run folder or a report artifact folder.

6. src/ - reusable modules and functions

Everything here should be importable, testable, and focused. Each module should own one piece of responsibility.

6.1 src/config.py

Function / Location	What it does	Why it exists	Used by
load_config(path)	Reads YAML or JSON config into a simple object or dictionary.	All scripts need a consistent way to load settings.	Used by prepare_pairs.py and run_eval.py.
resolve_paths(config)	Normalizes relative paths into project-safe paths.	Prevents path bugs and makes configs portable.	Used right after load_config().
validate_required_config_fields(config)	Checks that required keys exist before running.	Fail early instead of deep inside the pipeline.	Used by load_config() or the caller script.

6.2 src/io_utils.py

Function / Location	What it does	Why it exists	Used by
load_pairs_csv(path)	Loads saved pair artifacts into a dataframe.	Evaluation starts from a saved deterministic pair file.	Used by run_eval.py.
save_csv(df, path)	Writes dataframes like sweep tables or predictions.	Many modules need a consistent save helper.	Used by evaluation.py, tracking.py, error_analysis.py.
save_json(data, path)	Writes metrics, confusion matrix, run info, selected threshold, or metadata.	JSON is easy to inspect and reproducible.	Used by tracking.py, evaluation.py, and scripts.
copy_config_snapshot(config, path)	Saves the exact config used in a run folder.	Critical for reproducibility.	Used by run_eval.py when a run starts.

6.3 src/validation.py

Validation is part of the milestone. The idea is to fail early when inputs or outputs are malformed.

Function / Location	What it does	Why it exists	Used by
validate_pairs_df(df)	Checks required columns, missing values, label format, and split values.	Bad pair files should be caught before scoring or thresholding.	Used by run_eval.py after load_pairs_csv(); can also be used by prepare_pairs.py before saving.
validate_image_paths(df)	Confirms referenced image paths exist.	Prevents silent failures during embedding or score computation.	Used by validate_pairs_df() or directly by prepare_pairs.py.
validate_threshold_config(threshold_cfg)	Checks threshold mode, range, and fixed value format.	Thresholding is central to the milestone; invalid configs should fail early.	Used by run_eval.py before evaluation.

Function / Location	What it does	Why it exists	Used by
validate_scores_length(df, scores)	Ensures number of scores equals number of pairs.	A common pipeline bug is misaligned lengths.	Used by run_eval.py after scoring.
check_split_integrity(df)	Checks for leakage or invalid split assignments.	The milestone explicitly cares about fair non-test threshold selection.	Used by prepare_pairs.py and run_eval.py.

6.4 src/metrics.py

Keep metric formulas here so they are reusable and easy to test on toy inputs.

Function / Location	What it does	Why it exists	Used by
compute_confusion_counts(y_true, y_pred)	Returns TP, FP, TN, FN.	Many downstream metrics and the confusion matrix depend on these counts.	Used by evaluate_at_threshold(), report helpers, and tests.
compute_accuracy(...)	Computes standard accuracy.	Provides a common baseline metric.	Used by summarize_metrics().
compute_precision_recall_f1(...)	Computes precision, recall, and F1.	Threshold comparisons often rely on these.	Used by summarize_metrics() and threshold sweeps.
compute_balanced_accuracy(...)	Computes class-balanced accuracy.	A strong threshold-selection rule for verification tasks.	Used by summarize_metrics() and select_threshold().
summarize_metrics(y_true, y_pred)	Builds one dictionary of all metrics from predictions.	Centralizes metric generation so run outputs stay consistent.	Used by evaluate_at_threshold().

6.5 src/thresholding.py

This module isolates score-to-decision logic. That is important because you will run it many times under different thresholds.

Function / Location	What it does	Why it exists	Used by
apply_threshold(scores, threshold, higher_means_same)	Converts continuous scores into binary decisions.	This is the exact decision layer of the verifier.	Used by evaluate_at_threshold() and run_threshold_sweep().
generate_threshold_grid(cfg)	Creates the list of thresholds to test in sweep mode.	Keeps sweep configuration separate from evaluation logic.	Used by run_eval.py or run_threshold_sweep().
select_threshold(sweep_df, rule)	Chooses one operating threshold using a fixed rule such as max balanced accuracy.	The milestone requires deliberate threshold selection on a non-test split.	Used after run_threshold_sweep(); selected value is reused in locked runs.
is_higher_score_same_person()	Returns or checks score direction convention.	Prevents the very common bug of thresholding in the wrong direction.	Used by apply_threshold() and run_eval.py.

6.6 src/evaluation.py

This module should hold the higher-level evaluation logic that combines metrics and thresholding.

Function / Location	What it does	Why it exists	Used by
compute_similarity_scores(eval_df, scoring_cfg)	Runs your existing embedding and similarity pipeline on each pair.	Evaluation needs a score for each pair.	Used by run_eval.py when scores are not preloaded.
load_or_compute_scores(eval_df, scoring_cfg)	Loads saved scores if they exist, otherwise computes them.	Lets you keep the runner flexible without cluttering main().	Used by run_eval.py.
evaluate_at_threshold(eval_df, threshold, higher_means_same)	Applies threshold, computes predictions, metrics, and confusion counts.	Needed for locked-threshold runs and for final summaries after sweep selection.	Used by run_eval.py and can be reused by error-analysis prep.
run_threshold_sweep(eval_df, thresholds, higher_means_same)	Loops over thresholds and records metrics for each threshold.	You need this for the ROC-style plot and threshold selection.	Used by run_eval.py in sweep mode.
attach_predictions(eval_df, y_pred, threshold)	Adds prediction columns to the pair dataframe and records the threshold used.	Predictions should be saved for error analysis and debugging.	Used by evaluate_at_threshold() or run_eval.py before saving predictions.csv.

6.7 src/tracking.py

This module is how one evaluation attempt becomes a tracked run. Without this module, you are just printing metrics, not managing experiments.

Function / Location	What it does	Why it exists	Used by
make_run_id()	Creates a unique run identifier.	Every tracked run needs a stable name.	Used by run_eval.py at run start.
get_timestamp()	Returns the run time.	Supports ordering and traceability of runs.	Used by run_eval.py and save_run_info().
get_git_commit_hash()	Reads the current git commit if available.	Connects results to the exact code version.	Used by run_eval.py when building run metadata.
create_run_dir(root, run_id)	Creates a folder for this run under outputs/runs/.	You need one place to store all artifacts for the run.	Used by run_eval.py.
save_run_info(path, metadata)	Writes run_info.json with config name, note, split, pair version, commit hash, etc.	This is the provenance record for the run.	Used by run_eval.py.
append_run_summary(summary_csv, row)	Appends one compact row to the master tracking file.	Lets the grader compare runs quickly without opening every folder.	Used by run_eval.py at the end of each run.

6.8 src/plotting.py

Function / Location	What it does	Why it exists	Used by
plot_roc_style_curve(sweep_df, path)	Creates the ROC-style plot from sweep results.	The report explicitly needs a ROC-style figure.	Used by run_eval.py after sweep mode.
plot_threshold_vs_metric(sweep_df, metric, path)	Plots threshold against a metric such as balanced accuracy or F1.	Helps you explain threshold behavior during analysis.	Used optionally by run_eval.py or report prep.

6.9 src/error_analysis.py

This module turns saved predictions into the evidence required for the report.

Function / Location	What it does	Why it exists	Used by
extract_false_positives(pred_df)	Returns different-identity pairs predicted as same.	These are one of the two main failure types to inspect.	Used by build_error_slices() and report prep.
extract_false_negatives(pred_df)	Returns same-identity pairs predicted as different.	The other main failure type for inspection.	Used by build_error_slices() and report prep.
build_error_slices(pred_df)	Constructs defined subsets such as near-boundary errors, few-image identities, or flagged low-quality pairs.	The milestone wants at least two real slices, not random cherry-picked examples.	Used by run_error_analysis.py.

Function / Location	What it does	Why it exists	Used by
sample_representative_examples(slice_df, k)	Chooses a few examples per slice for the report.	You need examples that illustrate the pattern without dumping everything.	Used by run_error_analysis.py.
save_slice_summary(slice_stats, path)	Writes slice counts and notes to disk.	Makes error analysis reproducible and report-ready.	Used by run_error_analysis.py.

7. Pseudocode guide for the main scripts

7.1 Pseudocode for prepare_pairs.py

```
1. Parse config path.
2. Load config and validate required fields.
3. Load base records or image manifest from Milestone 1.
4. Build deterministic split map.
5. Construct positive and negative pairs under the chosen policy.
6. Attach labels and split names.
7. Run pair validation checks.
8. Save pairs CSV and pair metadata JSON.
9. Print pair version, output path, and basic counts.
```

7.2 Pseudocode for run_eval.py

```
1. Parse --config and optional --note.
2. Load config.
3. Create run_id, timestamp, and commit hash.
4. Create run directory.
5. Save config snapshot and run_info.json.
6. Load saved pairs CSV.
7. Validate pair schema, labels, split values, path existence, and threshold config.
8. Filter to the requested eval split.
9. Load or compute similarity scores.
10. Validate score length and numeric format.
11. If mode == sweep: run threshold sweep, save sweep CSV, select threshold, save threshold JSON,
create ROC-style plot.
12. Evaluate at the selected threshold and save metrics, confusion matrix, and predictions.
13. If mode == fixed: evaluate directly at fixed threshold and save metrics, confusion matrix, and
predictions.
14. Append one row to run_summary.csv.
15. Print final summary.
```

7.3 Pseudocode for run_error_analysis.py

```
1. Load predictions.csv from a specific run folder.
2. Extract false positives and false negatives.
3. Build at least two defined slices.
4. Compute counts and select representative examples.
5. Save slice tables and a summary JSON or CSV.
6. Use those outputs in the report.
```

8. Minimum tracked runs you should execute

- **Run 1:** baseline threshold sweep on validation.
- **Run 2:** baseline locked-threshold evaluation on validation.
- **Run 3:** baseline final test evaluation using the locked threshold.
- **Run 4:** improved pair version threshold sweep on validation.
- **Run 5:** improved final evaluation on validation or test using the same threshold-selection rule.

Why these five runs work

They show the full evaluation loop: baseline behavior, threshold choice, final reporting, one data-centric change, and fair post-change reevaluation.

9. What should be saved in each run folder

- **run_info.json** - metadata such as run id, timestamp, config name, pair version, split, note, and commit hash.

- **config_used.yaml** - exact config snapshot.
- **metrics.json** - final metrics at the selected threshold.
- **confusion_matrix.json** - TP, FP, TN, FN counts.
- **threshold_sweep.csv** - only for sweep runs.
- **selected_threshold.json** - threshold and selection rule.
- **roc_curve.png** - for the report.
- **predictions.csv** - labels, scores, predictions, threshold, and pair identifiers for later error analysis.

10. What the master log should contain

Keep one compact file such as **outputs/run_summary.csv**. Each row should record run_id, timestamp, commit_hash, config_name, split, pair_version, mode, selection_rule, selected_threshold, main metrics, and a short note.

11. Testing plan

Tests should be small and fast. The goal is reliability, not a giant test suite.

Function / Location	What it does	Why it exists	Used by
tests/test_metrics.py	Checks known toy cases for confusion counts and metric formulas.	Metric mistakes would poison every run.	Covers metrics.py.
tests/test_thresholding.py	Checks threshold application and threshold-direction logic.	Wrong threshold direction is a silent but critical bug.	Covers thresholding.py.
tests/test_validation.py	Checks rejection of missing columns, bad labels, and invalid threshold configs.	Validation is a required milestone component.	Covers validation.py.
tests/test_integration_eval.py	Runs a tiny end-to-end evaluation on a miniature fixture and checks expected output files.	Proves the pipeline works as a system from input pairs to saved artifacts.	Covers run_eval flow and core src modules together.

12. Report checklist

- Very brief reminder of the task and baseline system.
- Description of tracked-run process.
- ROC-style plot from validation sweep.
- Confusion matrix at the selected threshold.
- Short explanation of the threshold-selection rule.
- Before-vs-after comparison for the data-centric change.
- At least two error slices with example pairs and hypotheses.
- One or two closing sentences describing the most important lesson from the iteration.

13. Clean-clone and final packaging checklist

- README explains environment setup and gives copy-pastable run commands.
- All paths in configs and scripts work from a fresh clone.
- The main reported result can be reproduced from the README.
- Lightweight run evidence is present in the repository.
- The report path is obvious.
- Only after the clean-clone test passes should you create the milestone tag.

14. Final recommendation on implementation order

- **First:** config.py and io_utils.py - they unblock everything.
- **Second:** validation.py - fail early before debugging deeper code.

- **Third:** metrics.py and thresholding.py - the mathematical core.
- **Fourth:** tracking.py - now your experiments become tracked runs.
- **Fifth:** evaluation.py - connect scores, thresholds, and metrics.
- **Sixth:** prepare_pairs.py and run_eval.py - the actual commands you will run.
- **Seventh:** error_analysis.py and run_error_analysis.py.
- **Eighth:** tests, report, README, clean-clone validation, and final tag.

Bottom line

The milestone becomes manageable once you treat it as three separate pieces: **prepare deterministic pair artifacts**, **run tracked evaluations**, and **analyze failures**. Keep scripts thin, keep reusable logic in src/, and make every run leave behind artifacts that explain exactly what happened.